Алгоритм GEO-аудита сайта

Oct 31, 2026 · @Господин

Цель

По адресу сайта автоматически получить отчёт в вашем макете. Ошибки должны отсеиваться без участия человека, расходы должны быть минимальными. Исходные данные: адрес сайта, 40 вопросов с правками пользователя, ответы ChatGPT, Claude и Gemini с поиском, страницы сайта. Результат — `report.html`, где каждая цифра прослеживается до исходного ответа.

Разбор исходного алгоритма

Что не так	Что меняю	Почему
Цена сбора не управляется. Это 1200 ответов с поиском, и почти все деньги уходят сюда	**Адаптивные повторы**: 3 повтора, и добор до 10 только там, где ответы расходятся	Там, где компанию стабильно не называют или стабильно называют, 10 повторов не дают новой информации. Ожидаю около 700 ответов вместо 1200
В плане была ручная проверка 50 ответов	Её заменяет автоматическая выборочная проверка моделью другого семейства	Человек в алгоритме не участвует
Спорные случаи разбирала умная модель того же семейства. Ответы Claude размечал Claude	Спорные случаи и случайные 5 % ответов разбирает модель OpenAI или Google	Повторный ответ той же семьи моделей — не независимая проверка

Что не так	Что меняю	Почему
Поиск названия по точному совпадению	Учитываются падежные окончания и транслитерация	Иначе «Ромашкой» и «Romashka» не найдутся, и упоминаний будет меньше, чем на самом деле
Склейка названий конкурентов и типы источников были в коде, но не в алгоритме	Отдельный шаг: сначала код, спорное — дешёвая модель	Иначе «Альфа» и «Alfa» считаются разными конкурентами
Не было правил на отказы: сайт не читается, система не отвечает	Для каждого шага задан лимит повторов и запасной путь	Алгоритм должен дойти до отчёта сам
Ссылки Gemini — переадресация Google	Код её разворачивает, чтобы получить настоящий адрес	Тогда видно, какие страницы вашего сайта цитируются
Проверка «текст без скриптов» была на глаз	Браузер (Playwright) открывает только подозрительные страницы, до 5 штук	Точно и недорого
При адаптивных повторах простое «k из n» искажает итог	Общая доля = среднее по вопросам, у каждого вопроса одинаковый вес	Иначе вопросы с 10 повторами весили бы больше вопросов с 3 повторами
Haiku резко дорожает на длинных запросах	Пачки ответов ограничены по размеру	У Claude Haiku 5.5 запрос длиннее 100 000 токенов стоит $0.50 за миллион входа вместо $0.10 (источник)

Что изменилось из-за правки

Брендовых вопросов больше нет. Алгоритм изменён только в местах, которые от них зависели:

- **Шаг 2:** модель больше не составляет 6 вопросов о компании.
- **Шаг 3:** убран параллельный сбор брендовых ответов и их пересбор при смене названия.

- **Шаг «Сверка фактов о компании» удалён:** без брендовых ответов ему не на чем работать. Следующие шаги сдвинулись на один номер вверх.
- **Подсчёт и план:** из входа убран `claims.json`. Вместе с ним пропали находка о противоречиях в описании и шаг плана «уточните факт на сайте».
- **Разбор:** удалены две строки таблицы — про сверку фактов в один запрос и про запуск брендовых вопросов до правок пользователя.
- **Цена:** ответов стало примерно на 54 меньше, около 700 вместо 750. Диапазоны цены остаются прежними.

Пустым остаётся раздел 5 макета «Что нейросети рассказывают о вас — и верно ли это»: его данные брались из брендовых вопросов. Решение по нему — в открытых вопросах.

Шаги 0–3: сайт и вопросы

0. Ввод адреса

- **Исполнитель:** пользователь. Проверку делает Python (httpx).
- **Вход:** адрес сайта.
- **Действие:** код приводит адрес к виду `https://домен/`, открывает главную страницу и проверяет, что сайт не перенаправляет на чужой домен.
- **Выход:** проверенный адрес и итоговый домен.
- **Условие перехода:** главная отдала нормальный ответ (код 200) со страницей. Если нет, пользователь сразу видит причину: «сайт не открылся» или «сайт ведёт на другой домен — проверить его?». Анализ не запускается, деньги не тратятся.

1. Чтение сайта и технические проверки

- **Исполнитель:** Python: httpx, trafilatura, beautifulsoup4, protego. Playwright подключается только для страниц с подозрительно малым количеством текста.
- **Вход:** адрес из шага 0.
- **Действие:**
  - Код читает robots.txt, карту сайта и до 30 страниц. В первую очередь берутся «о компании», цены, услуги, контакты, вопросы и ответы.
  - Затем выполняются проверки по правилам: доступ 14 роботов ИИ-систем и поисковиков с номерами строк robots.txt; запреты индексации и показа фрагментов; одинаково ли сервер отвечает обычному браузеру и роботам; текст без скриптов; карта сайта и HTTPS.

    - Язык и страна определяются по разметке страницы, доменной зоне и контактам.
- **Выход:** `crawl.json` (страницы и их тексты) и `checks.json` (статус каждой проверки и основание).
- **Условие перехода:** прочитано не меньше 3 страниц, где больше 300 символов текста. Если меньше, код повторяет чтение с паузой и обычной подписью браузера. Не помогло — алгоритм просит пользователя описать компанию в 2–3 предложениях и дальше работает по этому описанию и главной странице. Проверка, которую не удалось выполнить, получает статус «нет данных», а не «пройдено».

2. Профиль компании и вопросы

- **Исполнитель:** Claude Sonnet 5.5, один запрос.
- **Вход:** тексты страниц (около 60 тысяч токенов), язык, страна, распределение вопросов по видам.
- **Действие:** модель составляет:
    - профиль: чем занимается компания, продукты, география, аудитория — у каждого факта цитата с сайта;
    - варианты названия;
    - 40 вопросов без названия компании.
- **Выход:** `profile.json` и `prompts.json`.
- **Условие перехода:** проверяет код.
    - Ответ соответствует заданной структуре.
    - Цитаты есть в текстах страниц.
    - Число вопросов каждого вида совпадает с заданным.
    - В вопросах нет названия компании (тем же поиском, что на шаге 5).
    - Нет почти одинаковых вопросов (rapidfuzz, сходство от 85 %).

  Если что-то нарушено: один повтор со списком нарушений; снова не так — годные вопросы остаются, недостающие догенерируются коротким отдельным запросом; и это не помогло — работаем с тем, что есть, и в отчёте пишем, сколько вопросов проверено. Факты без подтверждающей цитаты удаляются из профиля.

3. Пользователь смотрит вопросы

- **Исполнитель:** пользователь. Проверку правок делает Python.
- **Вход:** профиль и вопросы из шага 2.

- **Действие:** пользователь может поправить профиль и вопросы и добавить до 10 своих.
- **Выход:** итоговый `prompts.json`, где у каждого вопроса есть пометка: предложен, изменён или свой.
- **Условие перехода:** проверяет код.
  - Вопрос не пустой, длиной 5–40 слов, не повторяет другой, вид указан.
  - Если в вопросе есть название компании, он исключается из расчёта видимости, и пользователь сразу видит пояснение.

  Ошибки показываются пользователю в интерфейсе.

Шаги 4–8: сбор и разметка ответов

4. Сбор ответов

- **Исполнитель:** Python: asyncio, официальные библиотеки openai, anthropic и google-genai, tenacity для повторов. Какие именно модели измерять — открытый вопрос 1.
- **Вход:** вопросы, страна, язык.
- **Действие:**
  - Каждый вопрос отправляется без истории и без служебной инструкции, с включённым поиском и указанием страны. Число поисков на ответ ограничивается 2–3 там, где API это позволяет.
  - Адаптивные повторы: на каждую пару «вопрос × система» делается 3 повтора. Если во всех трёх результат одинаковый (назвали везде или нигде), повторы останавливаются. Если результаты разные, добираем до 10. Для этого решения код сразу ищет название в ответе (шаг 5).
  - Каждый ответ сохраняется дважды: полный ответ API, сжатый, с контрольной суммой; строка в `answers.jsonl`.
- **Выход:** папка `raw/` и файл `answers.jsonl`.
- **Условие перехода:**
  - Сбой сети или лимит — до 3 повторов с паузой.
  - API отклонил запрос — ответ помечается ошибкой и не повторяется.
  - Пустой ответ — 1 повтор.
  - Если у системы не получено больше 20 % ответов, она исключается из сравнения, и в отчёте объясняется почему.
  - Если не отвечает ни одна система, аудит останавливается, а пользователь получает сообщение.

Ответы с ошибкой не входят в подсчёт, но их число показывается в отчёте.

5. Механический разбор

- **Исполнитель:** Python: re, tldextract, httpx.
- **Вход:** `answers.jsonl`, варианты названия, домен.
- **Действие:**
  - Поиск названия с окончаниями (до 3 букв после основы) и в транслитерации.
  - Сбор доменов источников и проверка, есть ли среди них ваш сайт.
  - Разворачивание переадресаций Gemini.
  - Удаление ссылок из текста ответа: они уже лежат отдельно, а в тексте только занимают место.
- **Выход:** те же строки с новыми полями: где найдено название, домены источников, сослались ли на ваш сайт.
- **Условие перехода:** шаг детерминированный и проверки не требует. Если переадресация не развернулась, источник помечается «известен только домен».

6. Разметка ответов

- **Исполнитель:** Claude Haiku 5.5. Ответы идут пачками по 20–30, каждый запрос короче 100 тысяч токенов.
- **Вход:** тексты ответов, варианты названия, найденные кодом совпадения.
- **Действие:** для каждого ответа модель определяет:
  - найденное совпадение — это ваша компания: да, нет или не уверена;
  - вид ответа: список, связный текст, отказ или ответ не по теме;
  - какие бренды названы и в какой роли: рекомендован, упомянут или не советуют; место в списке, если ответ оформлен списком.

  К каждому решению прикладывается цитата.
- **Выход:** `annotations.jsonl`.
- **Условие перехода:** проверяет код.
  - Вернулись все ответы пачки, структура верна.
  - Каждая цитата дословно есть в ответе.
  - Каждый названный бренд встречается в тексте ответа.

  Если ошибка в отдельном пункте, пункт выбрасывается. Если сломана вся пачка: один повтор с текстом ошибки; затем пачка делится пополам, и попытка повторяется; затем ответы помечаются «не разобрано». Такие

ответы не участвуют в показателе «предложили как вариант». Факт упоминания, найденный кодом, для них остаётся. Если неразобранных ответов больше 10 %, этот показатель получает низкую надёжность.

7. Независимая выборочная проверка

- **Исполнитель:** дешёвая модель другого семейства. Основной вариант — GPT-6 Luna, запасной — Gemini Flash-Lite. Ключи к обоим уже нужны для сбора ответов.
- **Вход:** все случаи «не уверена» из шага 6 и случайные 5 % ответов (около 35). Разметку Haiku эта модель не видит.
- **Действие:** та же разметка, но только два поля: «это ваша компания?» и «роль бренда».
- **Выход:** `crosscheck.jsonl` и доля совпадений с Haiku.
- **Условие перехода:**
  - Случаи «не уверена» решает вторая модель. Если не уверена и она, упоминание не засчитывается: лучше недосчитать, чем приписать чужое.
  - Совпадение по выборке 90 % и выше — у показателей разметки средняя надёжность.
  - Совпадение ниже 90 % — «предложили как вариант» и роли конкурентов получают низкую надёжность, и в отчёте появляется предупреждение.

  Сама выборка ничего не исправляет, она только измеряет качество разметки.

8. Склейка названий конкурентов и типы источников

- **Исполнитель:** сначала Python (rapidfuzz, транслитерация), затем Claude Haiku 5.5 одним запросом.
- **Вход:** все бренды из шага 6 и самые частые домены из шага 5.
- **Действие:**
  - Код склеивает очевидные варианты: регистр, кавычки, латиница и кириллица, сходство от 90 %.
  - Оставшиеся спорные группы и домены уходят в Haiku. Она решает, объединять ли названия, и определяет тип домена: обзор, СМИ, каталог, форум, сайт компании и так далее.
- **Выход:** `brand_map.json` и `domain_types.json`.
- **Условие перехода:** проверяет код.
  - Каждое исходное название попало в какую-то группу.
  - Объединены только названия из присланного списка.

- Типы взяты из допустимого списка.

При ошибке — один повтор. Затем названия остаются несклеенными, а типы доменов получают значение «другое».

Шаги 9–12: подсчёт и отчёт

9. Подсчёт

- **Исполнитель:** Python: pandas, numpy.
- **Вход:** результаты шагов 4–8 и `checks.json`.
- **Действие:**
  - Доли по системам и видам вопросов, как среднее по вопросам.
  - Разброс оценки повторной выборкой по вопросам (bootstrap).
  - Устойчивость: назвали всегда, иногда или никогда.
  - Конкуренты и источники.
  - Есть ли вы в главных источниках: код открывает до 5 самых цитируемых сторонних страниц из ответов о выборе и ищет там вас и конкурентов.
- **Выход:** `aggregates.json`.
- **Условие перехода:** код сверяет итоги. Сумма ответов по системам должна равняться общему числу, а каждая доля — пересчитываться из исходных строк. Расхождение означает ошибку в коде: отчёт выходит без этого показателя и с пометкой.

10. Страницы для тем без ответа

- **Исполнитель:** Claude Sonnet 5.5. Шаг выполняется, только если такие темы есть.
- **Вход:**
  - вопросы, где вас не назвали ни разу, а конкурентов рекомендовали хотя бы в 30 % ответов;
  - список страниц сайта;
  - профиль компании.
- **Действие:** сгруппировать вопросы в 2–6 тем. Для каждой темы найти подходящую страницу сайта. Если её нет, предложить заголовок и 3–5 пунктов содержания строго в пределах того, что компания делает.
- **Выход:** `gaps.json`.
- **Условие перехода:** проверяет код.
  - Указанная страница есть среди прочитанных, цитата с неё дословная.

- В пунктах нет чисел и цен, которых нет в профиле.

Нарушение — пункт удаляется. Ошибка структуры — один повтор, затем шаг плана выводится без готового материала.

11. Находки и план действий

- **Исполнитель:** Python, правила и шаблоны фраз (Jinja2). Модели здесь не нужны.
- **Вход:** `aggregates.json`, `checks.json`, `gaps.json`.
- **Действие:**
  - 3–4 главные находки по фиксированным правилам.
  - План по уровням: «Неделя 1» — препятствия; «Недели 2–4» — пробелы; «Дальше» — внешние источники и гигиена сайта.
  - Внутри уровня шаги сортируются по числу затронутых вопросов.
  - У каждого шага есть основание, готовый материал и подсказка «как проверим».
- **Выход:** `findings.json` и `plan.json`.
- **Условие перехода:** у каждого шага есть ссылка на основание. Шаг без основания не создаётся.

12. Сборка отчёта и финальный контроль

- **Исполнитель:** Python (Jinja2) по вашему шаблону.
- **Вход:** результаты всех шагов.
- **Действие:** код подставляет данные в шаблон и проверяет:
  - каждое число на странице есть в `aggregates.json`;
  - пустые разделы показывают «нет данных» и причину;
  - в HTML нет внутренних инструкций (поиск по служебным меткам).
- **Выход:** `report.html`. PDF получается печатью из браузера.
- **Условие перехода:** все проверки пройдены. Если сломался один блок, отчёт собирается без него, с пометкой. Остальной отчёт не теряется.